



Práctica Final: Servicio de Envío de Mensajes

Asignatura: Sistemas Distribuidos

Grado en Ingeniería Informática

Curso Académico: 2025-2026

Alumna	NIA	Grupo Reducido
Montserrat Martis Contreras	100522371	81
Marta Veber	100522208	81

Indice

Primera Parte.....	2
1. Diseño del servidor.....	2
2. Diseño del cliente.....	3
3. Protocolo de comunicación.....	3
4. Implementación.....	4
Segunda Parte.....	6
1. Envío de ficheros adjuntos.....	6
2. Servicio web.....	7
3. Servicio RPC.....	8
Compilación y Ejecución.....	9
Batería de pruebas.....	10
1. Pruebas de la primera parte.....	10
2. Pruebas de la segunda parte.....	12
2.1. Pruebas de la Transferencia de Ficheros.....	12
2.2. Pruebas del Servicio Web.....	13
2.3. Pruebas de RPC.....	13
Conclusiones.....	14
Problemas encontrados.....	14
Opiniones.....	14

Primera Parte

En esta primera parte de la práctica hemos desarrollado un sistema de mensajería distribuido compuesto por un servidor y un cliente. El objetivo principal ha sido implementar un servicio que permita a varios usuarios registrarse, conectarse al sistema, consultar qué usuarios están activos y enviar mensajes entre ellos.

Dicho sistema sigue una arquitectura cliente-servidor, donde el servidor centraliza toda la lógica del sistema y los clientes actúan como interfaz de usuario. La comunicación se realiza mediante sockets TCP, utilizando un protocolo propio definido en el enunciado.

Además, se ha tenido en cuenta el carácter concurrente del sistema, permitiendo que múltiples clientes puedan interactuar con el servidor simultáneamente.

1. Diseño del servidor

El servidor es el componente central del sistema y se encarga de mantener toda la información necesaria para el funcionamiento del servicio. Para ello se ha definido una estructura *User* que representa a cada usuario registrado. En ella se almacenan su nombre (*username*), su estado de conexión (*connected*, siendo 1 para conectado y 0 desconectado), la información necesaria para poder comunicarse con él (IP y puerto) y los mensajes pendientes que aún no ha recibido (*struct Message*).

Para facilitar la gestión de usuarios, se han implementado varias funciones auxiliares, que encapsulan la lógica de acceso a la estructura global:

- *user_exists* permite comprobar si un usuario ya está registrado recorriendo la lista de usuarios.
- *find_user* devuelve el índice de un usuario dentro del array, lo que permite acceder directamente a su información.
- *remove_user* elimina un usuario desplazando los elementos del array.
- *add_user* añade un usuario nuevo al final del array.

Además, cada usuario mantiene su propio contador de mensajes enviados. Esto permite asignar identificadores únicos a los mensajes de cada remitente, como exige el protocolo.

El servidor mantiene todos los usuarios en una lista global. Dado que el servidor es concurrente y múltiples hilos pueden acceder a esta lista al mismo tiempo, se ha protegido su acceso mediante un mutex. De esta forma se evita que se produzcan inconsistencias cuando varios clientes realizan operaciones al mismo tiempo.

1.1. Modelo concurrente del servidor

El servidor está diseñado para atender múltiples clientes de forma concurrente. De forma que cada vez que se acepta una nueva conexión, se crea un hilo independiente encargado de

procesar esa petición.

Para la creación de hilos se utiliza la función *pthread_create*, pasando como argumento la función *handle_client*. Cada hilo recibe como parámetro el socket del cliente, lo que permite procesar de forma independiente la petición recibida. Además, tras crear el hilo, se utiliza *pthread_detach*, lo que permite liberar automáticamente los recursos del hilo cuando finaliza su ejecución.

La función *handle_client* es la encargada de atender una petición concreta del cliente. Este enfoque permite que varias operaciones (como registros, envíos de mensajes o consultas de usuarios) se realicen simultáneamente sin bloquear el servidor.

No obstante, al compartir datos globales entre hilos, ha sido necesario proteger las secciones críticas mediante un mecanismo de exclusión mutua. Esto garantiza que las operaciones sobre la lista de usuarios se realicen de forma segura.

2. Diseño del cliente

El cliente se ha implementado en Python y funciona mediante una interfaz de comandos en consola. Se ejecuta indicando la IP y el puerto del servidor. Cada comando introducido por el usuario se traduce en una operación del protocolo que se envía al servidor. Cada operación del cliente crea una nueva conexión TCP con el servidor mediante la función *connectToSolver*.

Una característica relevante del cliente es que utiliza un segundo hilo para escuchar mensajes entrantes. Esto permite que el usuario pueda seguir interactuando con la consola mientras recibe mensajes o confirmaciones.

2.1. Recepción de mensajes

Cuando un usuario se conecta, el cliente abre un socket local en un puerto libre y lanza un hilo de escucha. Este hilo queda esperando conexiones del servidor. Cuando recibe un mensaje, lo muestra por pantalla sin interrumpir la ejecución del programa. De esta forma se consigue un comportamiento similar al de aplicaciones reales de mensajería.

Este hilo utiliza un socket en modo escucha (*listen*) y acepta conexiones entrantes del servidor. Para evitar bloqueos permanentes, se utiliza un “timeout” en la llamada a *accept*, lo que permite comprobar periódicamente si el socket sigue activo. De esta forma, el hilo puede finalizar correctamente cuando el usuario se desconecta.

3. Protocolo de comunicación

La comunicación entre cliente y servidor se basa en el envío de cadenas de caracteres terminadas en `\0`. Por ello, en el servidor se ha implementado una función auxiliar

recv_string, que se utiliza para leer byte a byte todas las operaciones del protocolo hasta encontrar dicho carácter nulo. Este detalle es importante, ya que permite delimitar correctamente los mensajes recibidos por el socket. En el cliente se ha implementado una función equivalente, *_recv_string*.

Cada operación del sistema sigue un formato concreto. Por ejemplo, el cliente envía primero el nombre de la operación (como *REGISTER*, *CONNECT* o *SEND*) y después los parámetros necesarios.

El servidor responde siempre con un código numérico que indica el resultado de la operación, y en algunos casos envía información adicional, como el identificador de un mensaje o la lista de usuarios conectados.

4. Implementación

Registro de usuarios

La operación *REGISTER* permite añadir nuevos usuarios al sistema. El servidor comprueba si el nombre de usuario ya existe antes de registrarlo. Si el usuario no existe, se añade a la lista y se inicializan todos sus campos. En caso contrario, se devuelve un error indicando que nombre ya está en uso.

Eliminación de usuarios

La operación *UNREGISTER* elimina completamente a un usuario registrado del sistema. Antes de eliminarlo se eliminan también todos sus mensajes pendientes para evitar inconsistencias. La eliminación se realiza reorganizando la lista de usuarios, desplazando los elementos para no dejar huecos intermedios.

Conexión de usuarios

Cuando un usuario se conecta (*CONNECT*), el cliente envía al servidor el puerto en el que está escuchando. El servidor guarda esta información junto con la dirección IP del cliente. A partir de ese momento, el usuario queda marcado como conectado y puede recibir mensajes. Además, en el momento de la conexión, el servidor revisa si el usuario tiene mensajes pendientes y, en caso afirmativo, intenta enviarlos automáticamente.

Para realizar el envío de mensajes pendientes, el servidor crea un socket hacia el cliente utilizando la IP y el puerto almacenados previamente. A través de este socket se envía la operación *SEND MESSAGE* junto con los datos del mensaje.

Si el envío falla (por ejemplo, porque el cliente ya no está disponible), el servidor marca al usuario como desconectado y conserva el mensaje en la cola de pendientes. Esto permite reintentar la entrega en futuras conexiones.

Desconexión de usuarios

La desconexión (*DISCONNECT*) elimina la información de conexión del usuario, pero mantiene su registro en el sistema, permitiendo que el usuario vuelva a conectarse más adelante sin necesidad de registrarse de nuevo.

Envío de mensajes

El envío de mensajes (*SEND*) es la funcionalidad principal del sistema. Cuando un usuario envía un mensaje, el servidor: comprueba que tanto el remitente como el destinatario existen (en caso contrario se devuelve código 1), genera un identificador único para el mensaje y almacena el propio mensaje en la cola de pendientes del destinatario. Para evitar escribir fuera del array, se controla que no se supere el límite definido.

El almacenamiento de mensajes pendientes se realiza mediante un array dentro de la estructura del usuario. Se ha establecido un límite máximo de 100 mensajes por usuario para evitar desbordamientos de memoria.

Además, cuando el destinatario está conectado, el servidor intenta enviar el mensaje inmediatamente mediante una conexión directa. Si el envío es correcto, el mensaje se elimina de la cola de pendientes desplazando los elementos del array. En cambio, si no lo está, el mensaje se guarda para ser entregado más adelante, marcando al usuario como desconectado. Cuando un mensaje se entrega correctamente, el servidor envía una confirmación (*SEND MESS ACK*) al remitente.

Consulta de usuarios conectados

Esta operación *USERS* permite a un usuario obtener la lista de usuarios actualmente conectados. El servidor recibe el nombre del usuario que realiza la petición y comprueba si el usuario existe (en caso contrario devuelve código 2), si está conectado (en caso contrario devuelve código 1), devuelve código 0, y cuenta cuántos usuarios están conectados y envía tanto el número como sus nombres.

Segunda Parte

En esta segunda parte hemos ampliado la implementación del servicio de mensajería, incorporando el envío de ficheros entre usuarios, el empleo de un servicio web para normalizar los mensajes y un servicio de registro con RPC.

1. Envío de ficheros adjuntos

Antes de nada, para implementar este servicio era necesario que un usuario pueda acceder al ip y puerto del usuario que le ha enviado el archivo. Para ello, en el cliente se ha añadido una lista de usuarios conectados a sus atributos y se ha modificado la operación de consulta (USERS) para que al llamarla se actualice dicha lista.

Envío de un mensaje con fichero

La funcionalidad “SENDATTACH” permite a un usuario enviar un mensaje de texto junto con la referencia a un archivo a otro usuario del sistema. Es importante destacar que no se transfiere el archivo en sí, sino únicamente su ruta y nombre. La transferencia real del contenido del archivo se realiza posteriormente mediante el comando “GETFILE”. Esta parte funciona exactamente igual a “SEND”, con la clara excepción de los mensajes de confirmación y que el mensaje recibido tiene la ruta del fichero al final. Además, cabe destacar que hemos decidido implementar en el shell del cliente que la última palabra del comando sea el nombre del fichero.

Solicitud de transferencia de ficheros

La funcionalidad “GETFILE” permite a un usuario descargar un archivo que otro usuario ha compartido previamente. Esta operación establece una conexión directa (peer to peer) entre los dos usuarios, sin que el servidor intermedie en la transferencia del contenido del archivo.

El usuario que desea descargar el archivo invoca el comando, el cliente busca en su lista local de usuarios conectados la dirección IP y el puerto del emisor. Si no lo encuentra, refresca la lista pidiéndola al servidor con el comando “USERS” y vuelve a intentarlo. Una vez encontrada la dirección, el cliente crea un nuevo socket y se conecta directamente al emisor.

El hilo de escucha del cliente de origen recibe esta petición. Al recibir ‘GETFILE’, el cliente de origen se prepara para enviar el archivo. Para manejar archivos grandes de manera eficiente y no consumir toda la memoria, la transferencia se realiza por fragmentos (chunks): se obtiene el tamaño total del archivo y lo envía al cliente descargador como una cadena terminada en nulo, entra en un bucle donde lee el archivo en bloques (e.g., 1024 bytes) y envía cada bloque a través del socket. Este proceso continúa hasta que se ha enviado el archivo completo.

Tras solicitar el archivo, el cliente descargador recibe la cadena con el tamaño total del archivo y la convierte a un número entero. Luego, entra en un bucle de recepción: lee los datos del socket en bloques del mismo tamaño que los enviados por el emisor, cada bloque

recibido lo escribe en el archivo local. El bucle continúa hasta haber recibido la cantidad de bytes correspondiente al tamaño total del archivo.

Una vez completada la transferencia, ambos clientes cierran la conexión directa y muestran un mensaje de éxito en la consola.

2. Servicio web

En este apartado de la práctica hemos ampliado el sistema de mensajería mediante la incorporación de un servicio web encargado de procesar los mensajes antes de su envío. Este realiza una normalización del texto eliminando los espacios en blanco redundantes, garantizando que las palabras queden separadas por un único espacio. Se ha conseguido mantener la separación de responsabilidades entre componentes, sin necesidad de modificar el servidor de mensajería.

Para implementar esta funcionalidad se ha desarrollado un nuevo archivo independiente *web_service.py*, en el que se define la clase *MessageService*, que hereda de *BaseHTTPRequestHandler* y es la encargada de gestionar las peticiones HTTP recibidas. Dentro de esta clase, el método *do_POST* implementa la lógica principal del servicio, permitiendo procesar solicitudes dirigidas al endpoint */normalize*.

El funcionamiento consiste en leer el contenido del mensaje a partir del flujo de entrada de la petición. Una vez obtenido el texto, se aplica el proceso de normalización, que elimina automáticamente los espacios consecutivos. Finalmente, el servicio devuelve el resultado al cliente mediante una respuesta HTTP con código 200.

La inicialización del servicio se realiza a través de la función *run*, que crea una instancia de *HTTPServer* asociada a la clase *MessageService* y la pone en funcionamiento de forma indefinida. Este servicio se ejecuta como un proceso independiente en la máquina local del cliente, escuchando en el puerto 8080.

Modificaciones realizadas en el cliente

Para integrar el servicio web en el sistema, se han realizado modificaciones en el archivo *client.py*, concretamente en la lógica de envío de mensajes.

En primer lugar, se ha añadido una nueva función *normalize_message*, cuya responsabilidad es gestionar la comunicación con el servicio web. Utiliza la librería *requests* para realizar una petición HTTP POST, enviando el mensaje original como contenido de la petición y devolviendo el mensaje procesado recibido como respuesta.

Posteriormente, esta función se ha integrado dentro del método *send*, que implementa la operación *SEND* del protocolo. Antes de enviar el mensaje al servidor de mensajería, el cliente invoca a *normalize_message* para obtener la versión normalizada del texto. El resultado de esta llamada es el que finalmente se envía al servidor a través del socket TCP.

Además, dentro de la función *normalize_message* hemos incorporado un mecanismo de control de errores mediante una estructura *try-except*. Esto permite capturar posibles fallos en la conexión con el servicio web, como por ejemplo que el servicio no esté en ejecución. En estos casos, la función devuelve el mensaje original sin modificar, permitiendo que el sistema continúe funcionando con normalidad.

3. Servicio RPC

El servicio de logging se definió mediante el archivo de especificación RPC *log.x*, que describe el contrato de comunicación entre el cliente (*server.c*) y el servidor de logging (*log_server.c*).

En dicho archivo definimos la estructura *log_args* que encapsula los datos a registrar:

- *username*: Identificador del usuario que realizó la operación
- *op*: Tipo de operación ejecutada (REGISTER, SEND, SENDATTACH, etc.)
- *filename*: Nombre del archivo adjunto, en caso de aplicar SENDATTACH

El servidor de logging implementa la lógica del servicio RPC mediante la función *log_I_svc*. Esta función actúa como servidor RPC que recibe las solicitudes de logging e implementa la lógica: si la operación es SENDATTACH, incluye el nombre del archivo en el log; para otras operaciones, sólo registra el usuario y la operación. Por último, se llama a *fflush(stdout)* para asegurar que el output se imprima inmediatamente en lugar de quedar en buffer.

Modificaciones en server.c

Como el servidor del servicio principal de mensajería es el cliente del servidor RPC, no usamos la plantilla de cliente generada por *rpcgen*.

En *server.c* declaramos nuevas variables globales para su uso como cliente: *rpc_host* almacena la dirección LOG_RPC_IP, *rpc_client* es un puntero a la conexión persistente con el servidor RPC y *rpc_mutex* se usa para sincronizar acceso a la conexión RPC desde múltiples hilos. *rpc_host* y *rpc_client* son inicializados en el *main*.

La función *call_log_service* encapsula la lógica para invocar remotamente el servicio de logging. Utiliza un mutex para garantizar que solo un hilo accede a la conexión RPC en cada momento, previniendo condiciones de carrera. La función *log_I()* realiza la llamada remota al servidor. Al final del hilo *handle_client* se agrupan los argumentos en la estructura de *log_args* y se llama a *call_log_service*.

Compilación y Ejecución

Para compilar todo el proyecto se debe llamar a *make* en la terminal del directorio raíz. Este comando generará dos ejecutables:

- server: Servidor de mensajería
- log_server: Servidor de logging RPC

Antes de ejecutar client.py se debe ejecutar primero los tres otros componentes, cada uno en una terminal diferente.

1. Servicio Web

```
$ python3 web_service.py
```

2. Servicio RPC

```
./log_server
```

3. Servicio de mensajería

```
export LOG_RPC_IP=localhost  
./server -p <puerto servidor>
```

Después de ejecutar los servicios se puede ejecutar client.py:

```
python3 client.py -s <ip> -p <puerto servidor>
```

Cada cliente se debería ejecutar en una terminal diferente, y se puede interactuar con las operaciones como desee.

Batería de pruebas

1. Pruebas de la primera parte

Prueba general

En primer lugar, se arrancó el servidor en un puerto determinado, se lanzaron dos clientes en terminales distintas y se registraron dos usuarios. Ambos se conectaron correctamente, lo que permitió comprobar que el sistema gestiona adecuadamente el estado de conexión. Posteriormente, se verificó la operación de consulta de usuarios, confirmando que ambos aparecen como conectados.

Además, se realizó el envío de un mensaje entre los dos usuarios conectados. El destinatario recibió el mensaje de forma inmediata y el remitente recibió la confirmación correspondiente.

Después, se probó el caso en el que el destinatario estaba desconectado. El mensaje no se entregó inmediatamente, sino que quedó almacenado en el servidor. Finalmente, al volver a conectarse el destinatario, el mensaje pendiente fue entregado automáticamente, lo que confirma el correcto funcionamiento del sistema de almacenamiento y entrega diferida.

Estas son las salidas correspondientes de las tres terminales empleadas para esta prueba:

Servidor	Cliente 1 (marta)	Cliente 2 (montse)
<pre>./server -p 12345 s> init server 0.0.0.0:12345 s> REGISTER marta OK s> REGISTER montse OK s> CONNECT marta OK s> CONNECT montse OK s> CONNECTEDUSERS OK s> SEND MESSAGE 1 FROM marta TO montse s> DISCONNECT montse OK s> MESSAGE 2 FROM marta TO montse STORED s> SEND MESSAGE 2 FROM marta TO montse s> CONNECT montse OK s> SEND MESSAGE 2 FROM marta TO montse s> DISCONNECT marta OK s> DISCONNECT montse OK</pre>	<pre>python3 client.py -s 127.0.0.1 -p 12345 c> REGISTER marta REGISTER OK c> CONNECT marta CONNECT OK c> USERS CONNECTED USERS (2 users connected) OK marta montse c> SEND montse Hola Montse SEND OK - MESSAGE 1 c> SEND MESSAGE 1 OK c> SEND montse Mensaje offline SEND OK - MESSAGE 2 c> SEND MESSAGE 2 OK c> DISCONNECT marta DISCONNECT OK c> QUIT +++ FINISHED +++</pre>	<pre>python3 client.py -s 127.0.0.1 -p 12345 c> REGISTER montse REGISTER OK c> CONNECT montse CONNECT OK s> MESSAGE 1 FROM marta Hola Montse END c> DISCONNECT montse DISCONNECT OK c> CONNECT montse CONNECT OK s> MESSAGE 2 FROM marta Mensaje offline END c> DISCONNECT montse DISCONNECT OK c> QUIT +++ FINISHED +++</pre>

Prueba de múltiples mensajes

Hemos decidido que es importante comprobar el correcto incremento de IDs, el orden de entrega y la gestión de la cola de mensajes. Para ello se han vuelto a registrar dos usuarios.

En el primer caso, con ambos usuarios conectados, se han enviado 4 mensajes consecutivos con ambos usuarios conectados. Mientras que en el segundo caso, antes de enviar tres mensajes más se ha desconectado a uno de los clientes para, al volver a conectarlo, comprobar si se envían los mensajes que estaban en espera de forma seguida y ordenada.

Efectivamente, el sistema ha respondido de la forma esperada. Los 4 primeros mensajes se han enviado al instante, en cambio, los 3 últimos mensajes se han guardado en la cola de espera (indicándose respectivamente en el servidor con *MESSAGE 5 FROM marta TO montse STORED*, igual para el 6 y 7) y en el momento en el que el receptor se ha vuelto a conectar, le han llegado automáticamente, indicándose de nuevo en el servidor. Es importante destacar que los 7 mensajes han mantenido los IDs en orden correcto.

Pruebas de error

Por último, hemos considerado relevante observar cómo reacciona el sistema implementado ante casos de error.

- En primer lugar, hemos tratado de enviar un mensaje a un usuario inexistente, el cual ni se ha registrado ni se ha conectado. La respuesta del sistema ha sido la correcta:

En cliente: c> SEND user hola
SEND FAIL, USER DOES NOT EXIST

En servidor: s> SEND FAIL

- En segundo lugar, hemos comprobado que al conectar un usuario que no ha sido registrado previamente el sistema avisa de ello:

En cliente: c> CONNECT usuario
CONNECT FAIL, USER DOES NOT EXIST

En servidor: s> CONNECT usuario FAIL

- En tercer lugar, hemos intentado introducir los comandos con errores de escritura:

En cliente: c> CONECT marta
Error: command CONECT not valid

- Por último, hemos probado la operación *USERS* con el usuario desconectado previamente:

En cliente: c> USERS
ERROR: NOT CONNECTED

Como podemos observar, los errores se gestionan correctamente, por tanto, tras esta y las pruebas anteriores, se puede deducir que el sistema responde de la forma esperada y exitosa en todo tipo de situaciones previstas por el enunciado.

2. Pruebas de la segunda parte

2.1. Pruebas de la Transferencia de Ficheros

SENDATTACH

Como la funcionalidad de esta parte es idéntica a SEND, vamos a realizar las mismas pruebas de los caminos válidos. Estas son las salidas correspondientes de las terminales relevantes para esta prueba:

Servidor	Cliente 1 (montse)	Cliente 2 (marta)
<pre>s> REGISTER montse OK s> CONNECT montse OK s> REGISTER marta OK s> CONNECT marta OK s> SENDATTACH MESSAGE 1 FROM montse TO marta FILE test.txt s> DISCONNECT marta OK s> MESSAGE 2 FROM montse TO marta FILE test.txt STORED s> CONNECTEDUSERS OK s> CONNECT marta OK s> SENDATTACH MESSAGE 2 FROM montse TO marta FILE test.txt</pre>	<pre>c> REGISTER montse REGISTER OK c> CONNECT montse CONNECT OK c> SENDATTACH marta prueba 1 test.txt SENDATTACH OK - MESSAGE 1 c> SENDATTACH MESSAGE 1 test.txt OK c> SENDATTACH marta prueba 2 test.txt SENDATTACH OK - MESSAGE c> USERS CONNECTED USERS (1 users connected) OK montse :: 127.0.0.1 :: 49117 c> SENDATTACH MESSAGE 2 test.txt OK</pre>	<pre>c> REGISTER marta REGISTER OK c> CONNECT marta CONNECT OK c> s> MESSAGE 1 FROM montse prueba 1 END FILE test.txt c> DISCONNECT marta DISCONNECT OK c> CONNECT Syntax error. Usage: CONNECT <userName> c> CONNECT marta CONNECT OK s> MESSAGE 2 FROM montse prueba 2 END FILE test.txt</pre>

Como se puede apreciar, la funcionalidad de sendattach funciona correctamente, enviando los mensajes con archivos adjuntos esté el receptor conectado o desconectado.

GETFILE

Esta parte funciona de manera peer to peer, el servidor no interviene. Para las pruebas vamos a usar dos ficheros uno más pequeño, con solo un párrafo de texto, y uno más grande de 15 párrafos. Ambos fueron generados con lorem ipsum generator (<https://www.lipsum.com/>). Estas son las salidas correspondientes de las terminales relevantes para esta prueba:

Cliente 1 (montse)	Cliente 2 (marta)
<pre>c> SENDATTACH marta prueba con fichero pequeño small.txt SENDATTACH OK - MESSAGE 1 c> SENDATTACH MESSAGE 1 small.txt OK c> c> GETFILE small.txt REQUEST FROM marta c> c> FILE small.txt SENT TO marta c> SENDATTACH marta prueba con fichero grande big.txt SENDATTACH OK - MESSAGE 2 c> SENDATTACH MESSAGE 2 big.txt OK</pre>	<pre>c> s> MESSAGE 1 FROM montse prueba con fichero pequeÃ±o END FILE small.txt c> GETFILE montse small.txt marta1.txt ERROR: User not found, refreshing user list... CONNECTED USERS (2 users connected) OK montse :: 127.0.0.1 :: 60245 marta :: 127.0.0.1 :: 33719 FILE small.txt RECEIVED AND SAVED AS marta1.txt c></pre>

<pre>c> c> GETFILE big.txt REQUEST FROM marta c> c> FILE big.txt SENT TO marta c></pre>	<pre>s> MESSAGE 2 FROM montse prueba con fichero grande END FILE big.txt c> GETFILE montse big.txt marta2.txt FILE big.txt RECEIVED AND SAVED AS marta2.txt c></pre>
--	---

Al abrir los archivos locales marta1.txt y marta2.txt comprobamos que se ha copiado la información correctamente.

2.2. Pruebas del Servicio Web

Para este apartado hemos realizado una prueba completa (3 pasos) utilizando dos clientes y el servidor de mensajería. Se ha iniciado el servicio web, el servidor de mensajería y dos instancias de cliente, registrando y conectando dos usuarios. Las variaciones llevadas a cabo, demuestran que el sistema funciona correctamente tanto con como sin el servicio web.

En primer lugar, se ha realizado el envío de un mensaje desde un cliente que contenía múltiples espacios consecutivos entre palabras. El resultado observado ha sido que el destinatario recibe el mensaje correctamente formateado. Además, hemos probado desconectar al receptor antes de enviar un mensaje (operación de *MESSAGE STORED*). Al reconectarlo, el mensaje también ha llegado de forma normalizada e instantánea.

Por otro lado, se ha evaluado el comportamiento del sistema ante la caída del servicio web. Para ello hemos detenido el servicio manualmente mientras los clientes permanecían en ejecución. Al realizar un nuevo envío de mensaje con espacios extra, la función *normalize_message* ha detectado un error de conexión y se lo ha informado al emisor. Sin embargo, como era de esperar, el método *SEND* ha continuado su ejecución utilizando el mensaje original (sin procesado), enviándolo correctamente al servidor de mensajería. El destinatario ha recibido el mensaje sin normalizar.

2.3. Pruebas de RPC

Para probar el servicio de logging, simplemente realizamos las pruebas de la sección de transferencia de ficheros con el *log_server* ejecutado. Estas son las salidas correspondientes de las terminales relevantes para esta prueba:

Servidor Mensajería	Servidor Log
<pre>s> REGISTER montse OK s> CONNECT montse OK s> REGISTER marta OK s> CONNECT marta OK s> SENDATTACH MESSAGE 1 FROM montse TO marta FILE small.txt s> CONNECTEDUSERS OK s> SENDATTACH MESSAGE 2 FROM montse TO marta FILE big.txt</pre>	<pre>LOG: montse REGISTER LOG: montse CONNECT LOG: marta REGISTER LOG: marta CONNECT LOG: montse SENDATTACH small.txt LOG: marta USERS LOG: montse SENDATTACH big.txt</pre>

Conclusiones

Problemas encontrados

Durante el desarrollo de la práctica nos hemos encontrado con varios desafíos que han requerido un análisis más detallado del que esperábamos inicialmente.

Uno de ellos fue la integración del servicio RPC. Dado que el servidor de mensajería actúa como cliente RPC y atiende múltiples hilos concurrentes, fue necesario gestionar correctamente el acceso al cliente RPC compartido. La solución adoptada fue mantener una conexión persistente protegida por un mutex dedicado, evitando el coste de crear y destruir la conexión en cada llamada.

En la transferencia de ficheros mediante GETFILE, la principal dificultad fue gestionar correctamente archivos de tamaño variable. Optamos por enviar primero el tamaño total del fichero y transferir el contenido en bloques, lo que permitió manejar tanto ficheros pequeños como grandes de forma eficiente sin consumir memoria innecesariamente.

Por otro lado, durante la implementación de la primera parte surgieron algunos problemas relacionados con la gestión de mensajes pendientes. Estos no se eliminaban correctamente tras ser entregados al destinatario, lo que provocaba duplicados cuando este volvía a conectarse. La solución consistió en reorganizar la cola de mensajes pendientes y actualizar el contador asociado después de cada entrega satisfactoria.

Opiniones

La realización de esta práctica ha sido una experiencia muy completa para afianzar los conceptos teóricos de la asignatura. Implementar desde cero un sistema distribuido real, con todas sus partes interactuando — sockets TCP, servicios web, RPC y transferencia de ficheros — nos ha permitido comprender en profundidad los retos que aparecen en este tipo de sistemas: la concurrencia, la sincronización, la gestión de errores en red y la comunicación entre componentes.

En general, consideramos que la práctica está bien diseñada para cubrir los principales paradigmas de los sistemas distribuidos estudiados en la asignatura, y estamos satisfechas con el resultado final obtenido.